

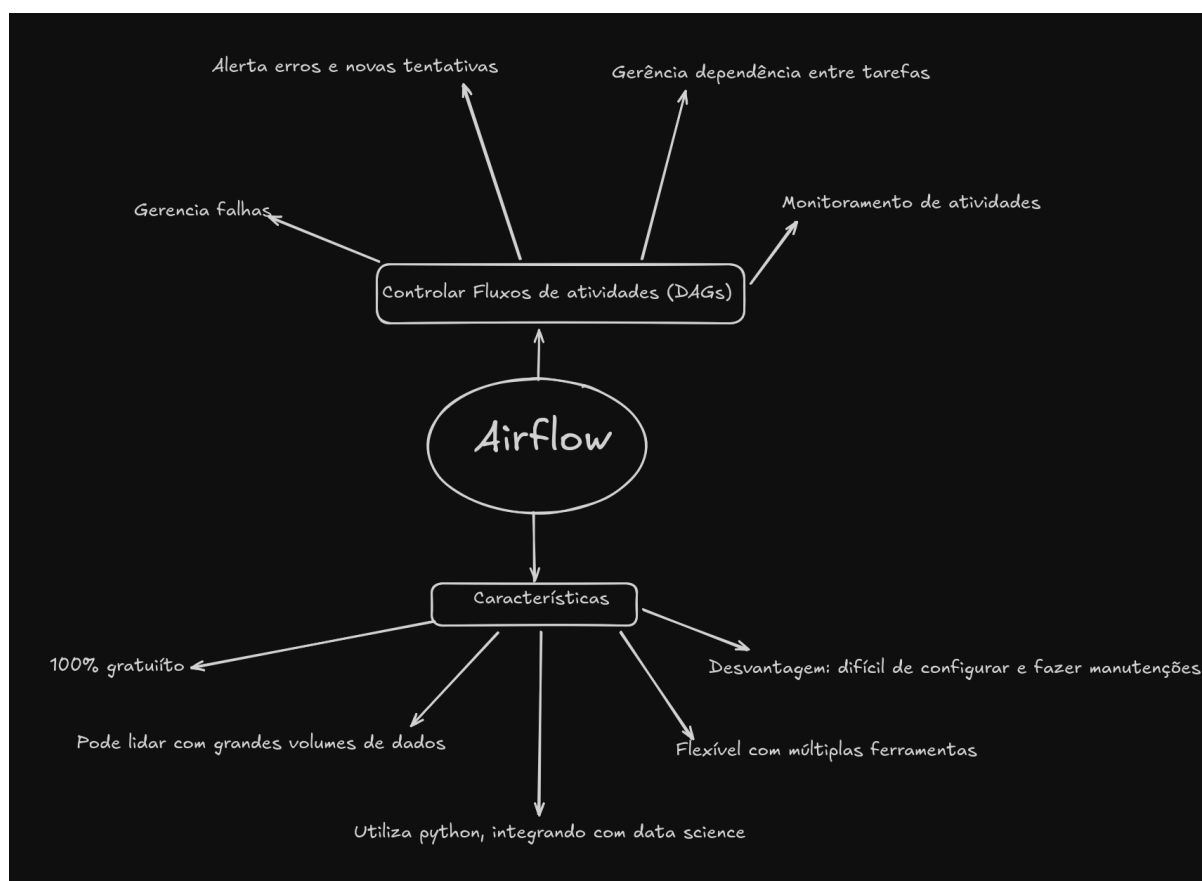
Relatório 12 - Pipelines de Dados I - Airflow (II)

Felipe Fonseca

1. Introdução

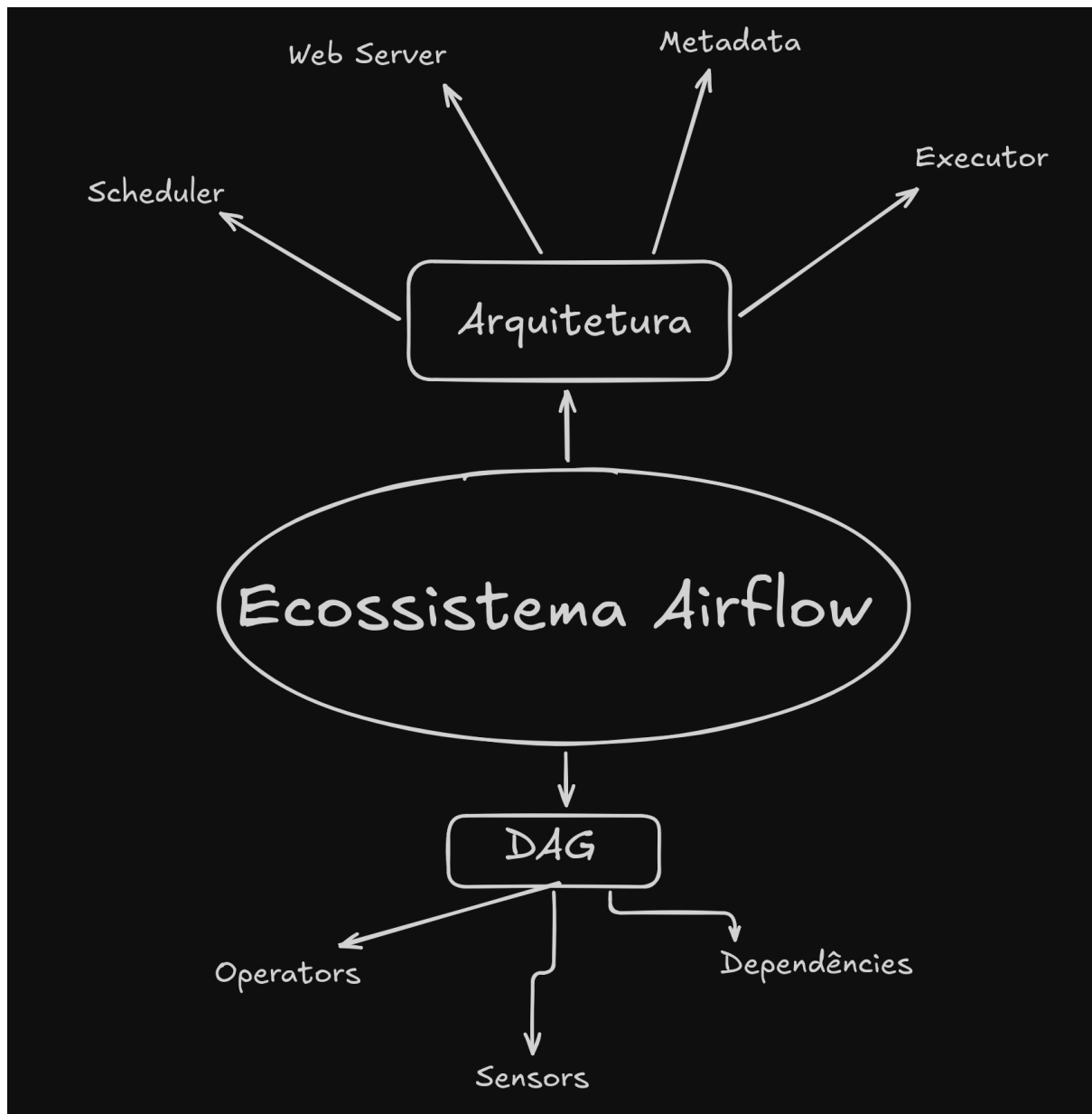
O objetivo dessa aula prática foi aprender sobre a implementação de pipeline de dados utilizando o Airflow em um container do Docker. Durante o processo, criamos algumas DAGs utilizando os conceitos de agendamento, gerenciando casos de falhas, fazendo testes e criando todo o fluxo de dados. O esperado era compreender como funciona o Apache Airflow e como utilizá-lo.

2. Desenvolvimento



O airflow é uma plataforma grátis criada para controlar tarefas e fluxos de códigos, sendo muito utilizada na área de ciência de dados devido a sua integração com códigos python. Ele é feito para lidar com problemas entre as dependências das tarefas, cuidar de problemas com falhas, além de fornecer métodos para fazer tentativas e alertar quando algo acontece. Uma das funções mais interessantes do Airflow é o agendamento de trabalhos, onde é definido quando as tarefas serão executadas, na primeira vez e depois o intervalo entre as execuções. Além disso, o

airflow permite uma visualização através da WEB, facilitando o entendimento do que está acontecendo para o usuário.



DAG, ou Directed Acyclic Graph é um grafo que sempre executa as atividades seguindo em frente, evitando um loop infinito. Ele é o responsável por implementar a execução das tarefas e garantir a sequência delas. A DAG também é a responsável por verificar e executar as tarefas novamente em caso de erro.

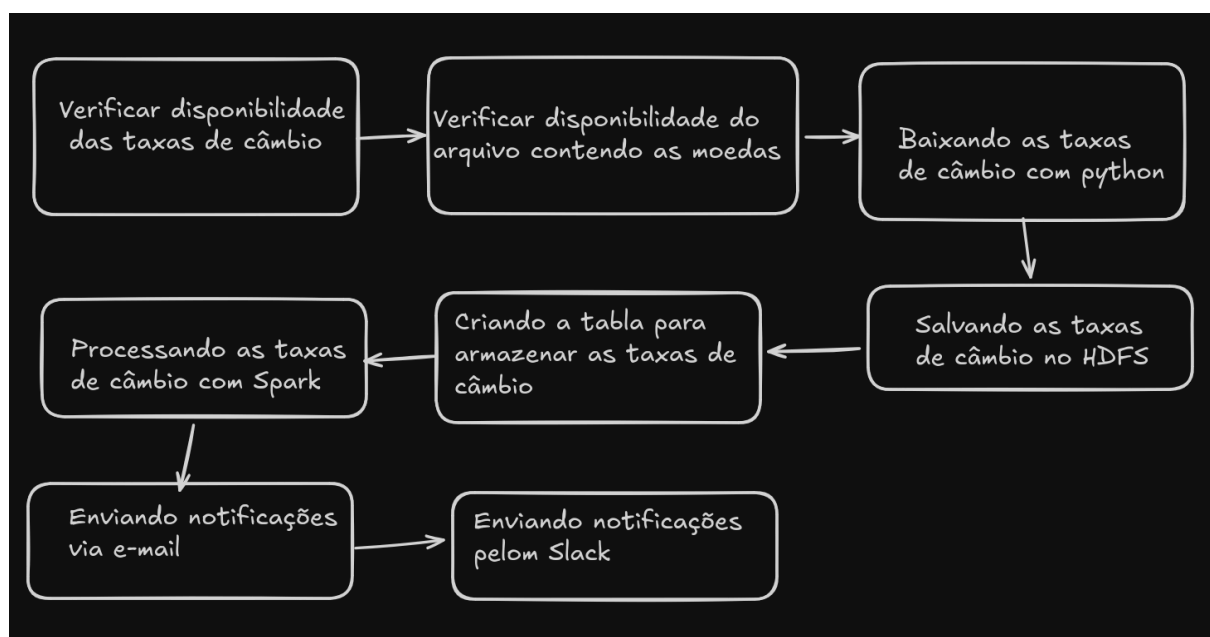
Outros componentes do Airflow são o Scheduler, ou seja, o responsável por agendar as tarefas e enviá-las para execução, o Metadata que é um banco de dados que armazena o histórico e as informações das tarefas, o Executor, que define como as tarefas serão processadas, e o Web Server, que é a interface que permite o monitoramento de forma visual e possui ferramentas de debug para a pipeline de dados.

A implementação da aula utilizou o Docker, para criar containers e isolar o Airflow e suas dependências, de forma que não desse erros ou conflitos com o computador.

A parte que teve maior foco na aula foi a implementação de um fluxo sobre um forex pipeline, utilizando operadores diversos:

- Http Sensor: monitora a api, foi usada para esperar a API das taxas de câmbio ficarem disponíveis antes de prosseguir.
- File Sensor: monitora arquivos, foi utilizado para verificar se existia um arquivo csv de moedas no sistema de arquivos.
- Python Operator: executa um script python, foi utilizado para executar a função de download e filtrar os dados (função própria do usuário, e não do airflow).
- Bash Operator: faz comandos de terminal no sistema, foi utilizado para criar diretórios e mover arquivos para o HDFS.
- Hive Operator: serve para criar tabelas, foi utilizado para criar a tabela do Data Warehouse para análise.
- Spark Submit Operator: serve para processamento pesado, foi utilizado para executar um script chamado `forex_processing.py`. que continha o processamento das taxas de câmbio.
- Email Operator: serve para enviar e-mails, foi utilizado para enviar uma mensagem aleatória de teste para um e-mail.
- Slack API Post Operator: serve para enviar mensagens em um ambiente slack, e foi utilizado para enviar mensagens em um canal em um ambiente criado para teste.

A ideia é que esse DAG siga o seguinte fluxo:



(Diagrama pego da aula, apenas traduzi)

E por último foi implementado alguns outros testes de controle de tempo ou manipulação e boas práticas em relação às DAGs:

- Alert Dag: implementada para testar os atributos de delay, time out, e callback, tanto em caso de sucesso quanto em caso de erro.
- Backfill: O catchup que vem por padrão habilitado faz com que o Airflow execute coisas pendentes do passado, desabilitar o catchup é uma boa prática, pois só faz sentido mantê-lo ativado caso realmente se deseje processar dados antigos ao ativar uma nova DAG.
- Depends Dag: foi implementado um teste com 3 tarefas, onde testamos dependência entre eles com o atributo “depends on past”, que quando utilizado faz com que a tarefa só seja executada se a anterior foi bem sucedida, e também testamos o parâmetro “wait for downstream”, que só marca a tarefa como concluída se todas as outras em espera também forem marcadas como concluída.
- Packaged Dag: foi testado a possibilidade de criar um zip contendo uma dag repartida em mais de um arquivo, isso é, um arquivo continha a definição da dag e as tarefas, e em outro arquivo, era contido as funções chamadas, e juntamos ambas em apenas um arquivo zip, que mostra utilidade para quando uma DAG for muito grande.
- Start and Schedule: foi testando como funciona o agendamento das dags e suas execuções em relação ao start date e o intervalo de tempo. Graças a isso foi possível observar que a primeira execução não é feita no start date, mas sim no start date + intervalo de tempo.
- Time zone: foi testando a importância de definir as datas de acordo com um fuso horário especificado, ao fazer isso, quando definido o horário, ele sempre levará em conta o fuso horário para fazer a execução, independente do horário que esteja na máquina ou do local onde o usuário está.

Testes:

- Dag validation: várias funções para verificar erros na sintaxe e loop infinito.
- Dag Definition: serve para validar se as tarefas e as dependências estão iguais a do projeto original.

3. Conclusão

O Airflow é muito útil para lidar com a organização de pipelines de dados e fluxos muito complexos, além de garantir que as execuções ocorram em uma ordem específica, sendo completamente monitorado e oferecendo controle em casos de erros, através de notificações e novas tentativas. Essa aula foi muito boa ao oferecer um container pronto já contendo processos rodando, o que simula muito bem um cenário real ajudando na integração ao aprendizado. O principal aprendizado que tive foi na utilização dos operadores e sensores das DAGs, porém

tive dificuldades ao tentar utilizar o Docker, ferramenta que ainda não me acostumei a utilizar.

4. Referências

Sessões 1 à 4 do curso “Apache Airflow: The Hands-On Guide”
Apache

Blog ds academy: [Apache Airflow: Características, Vantagens e Desvantagens no Cenário da Engenharia de Dados](#)

Datacamp: [O que é um DAG? Um guia prático com exemplos](#)